



Activity Prediction of Business Process Instances with Inception CNN Models

Nicola Di Mauro¹(✉), Annalisa Appice¹, and Teresa M. A. Basile^{2,3}

¹ Department of Computer Science, University of Bari “Aldo Moro”, Bari, Italy
nicola.dimauro@uniba.it

² Department of Physics, University of Bari “Aldo Moro”, Bari, Italy

³ Bari Division, National Institute for Nuclear Physics (INFN), Bari, Italy

Abstract. Predicting the next activity of a running execution trace of a business process represents a challenging task in process mining. The problem has been already tackled by using different machine learning approaches. Among them, deep artificial neural networks architectures suited for sequential data, such as recurrent neural networks (RNNs), recently achieved the state of the art results. However, convolutional neural networks (CNNs) architectures can outperform RNNs on tasks for sequence modeling, such as machine translation. In this paper we investigate the use of stacked inception CNN modules for the next-activity prediction problem. The proposed neural network architecture leads to better results when compared to RNNs architectures both in terms of computational efficiency and prediction accuracy on different real-world datasets.

Keywords: Business process monitoring · Sequence prediction · Deep learning

1 Introduction

Process mining [1] refers to the analysis of executions, often called traces, of a business process. In this research area, a challenging task is represented by the learning of probabilistic predictive models [20]. When learned, these models are then exploitable on incomplete or running traces, in order to predict missing events or forecast the evolution of running traces based on patterns extracted from a historical event log. One example of these predictive models includes techniques able to predict the next activity [2, 6, 11, 24]. This prediction ability may be considered to guarantee the higher utilization by acting proactively in anticipation.

In recent years, with the increasing popularity of deep artificial neural networks (NNs) [15] in many fields, such as image classification [18, 22], automatic speech recognition [9] and natural language processing [10], growing interest has arisen in using deep learning to analyze event logs to gain accurate insights into the future of a business process.

Recurrent neural networks (RNNs) are a kind of NNs specialized to deal with sequential data using internal memory and sharing parameters across different parts of the model. The authors in [11] presented one of the first application of deep NNs, adopting RNNs with long short-term memory (LSTM) cells [16], to the problem of predicting the next process activity. In [24] the authors proposed a new LSTM architecture for the next-activity prediction problem, which gains in accuracy compared to the architecture already reported in [11].

The recent systematic study available in [23] has compared different neural networks architectures, such as generic RNNs, and specialized RNNs with LSTM and GRU units [8], designed for the next-activity prediction problem. They have proved that all the neural networks architectures outperform other classical approaches.

Techniques based on LSTM architectures, suited for predicting the sequence of future activities of an ongoing case, could be improved by leveraging the knowledge about the structure of the process execution traces as well as the *a-priori* knowledge about future development, as already proven in [12].

In order to deal with sequential data, a valid alternative to RNNs could be the use of convolutional neural networks (CNNs) [21]. The same peculiar properties of CNNs—extracting features from local input patches and allowing for representation modularity—making them excel at computer vision could be also effective for sequence processing, where time could be considered as a spatial dimension. Such one-dimensional CNNs are competitive with RNNs at a cheaper computational cost. Instead, as recently showed in [3], simple convolutional architecture outperforms canonical recurrent networks such as LSTMs across a diverse range of tasks, while demonstrating longer effective memory.

However, a classical problem with CNNs is to correctly set the kernel size. The problem has been tackled in [22] where the authors proposed an inception module that simultaneously applies many convolutions with different kernel size to the same input—an optimal local sparse structure in a convolutional network can be approximated and covered by readily available dense components.

The aim of this paper is to investigate whether the use of one-dimensional CNNs could be competitive to RNNs for the next activity prediction problem. To this purpose we define an inception architecture for sequential data where inception modules of one-dimensional convolutions are stacked on top of each other. The proposed model, like other machine learning algorithms, is able to return the probability distribution over all the possible activities for the next event given an incomplete/running trace. The empirical evaluation, performed on real-world event logs, proves the effectiveness of the proposed neural network architecture both in term of prediction accuracy and computational cost.

Other approaches, such as that proposed in [20], deal with the problem of predicting the *time-to-completion* of running business process instances. Even if our proposed method can be extended to deal with this regression task, here we focus on the next activity prediction problem.

The background definitions are reported in the following section, while the formulated inception architecture is described in Sect. 3. The findings in the

evaluation of the proposed architecture are discussed in Sect. 4. Finally, Sect. 5 refocuses on the purpose of the research and draws the conclusions.

2 Next Activity Prediction Problem

In this section we provide some background definitions regarding the next-activity prediction problem.

2.1 Background

Process mining techniques aim to extract useful information from event logs, assuming that each event of a business process refers to an activity of a particular case—process trace. Further information may be linked to a specific event, such as the timestamp and other attributes. Here we define some useful process mining concepts.

Let $\mathcal{A}$ be the set of all possible activities, $\mathcal{C}$ the set of case identifiers, and $\mathcal{D}_i$ the set of attributes, $1 \leq i \leq m$. Let $\mathcal{E}$ be the set of events in a trace.

Definition 1 (Event). *An event $e_i \in \mathcal{E}$ is as a tuple $e_i = (a_i, c_i, t_i, d_{i1}, \dots, d_{im})$ where $a_i \in \mathcal{A}$, $c_i \in \mathcal{C}$, t_i is the timestamp, and $d_{ij} \in \mathcal{D}_j$, $1 \leq j \leq m$, are additional attributes.*

Given an event $e_i = (a_i, c_i, t_i, d_{i1}, \dots, d_{im})$, it is possible to define the functions $\pi_{\mathcal{A}}(e_i) = a_i$, $\pi_{\mathcal{C}}(e_i) = c_i$, $\pi_{\mathcal{T}}(e_i) = t_i$, and $\pi_{\mathcal{D}_j}(e_i) = d_j$ for each $1 \leq j \leq m$. As in [24] the additional attributes will not be considered in the following of this paper, thus leading to events characterised by the activity, the timestamp and the case identifier.

Definition 2 (Trace). *A trace is a finite sequence of events $\sigma = \langle e_1, e_2, \dots, e_n \rangle$, where each $e_i \in \mathcal{E}$, $n = |\sigma|$, and such that $\pi_{\mathcal{T}}(e_i) \leq \pi_{\mathcal{T}}(e_{i+1})$ and $\pi_{\mathcal{C}}(e_i) = \pi_{\mathcal{C}}(e_{i+1})$, $\forall 1 \leq i \leq n - 1$.*

Given the trace $\sigma = \langle e_1, e_2, \dots, e_n \rangle$, a *prefix* σ^k of σ , with $k = |\sigma^k| \leq |\sigma|$, is the trace $\sigma^k = \langle e_1, e_2, \dots, e_k \rangle$. In particular a trace is a complete process instance—started and ended—while a prefix is an instance in execution—running trace.

Definition 3 (Event log). *An event log is a set of traces such that each event appears at most once in the entire log.*

Table 1 reports a part of the event log described in the *Receipt phase* dataset adopted in the following experimental evaluation.

Table 1. Two traces of the event log dataset Receipt phase.

Event	Case ID	Activity	Timestamp
e_1	10011	Confirmation of receipt	2011/10/11 13:45:40.276
e_2	10011	Check confirmation of receipt	2011/10/12 08:26:25.398
e_3	10011	Adjust confirmation of receipt	2011/11/24 15:36:51.302
e_4	10011	Check confirmation of receipt	2011/11/24 15:37:16.553
e_5	10017	Confirmation of receipt	2011/10/18 13:46:39.679
e_6	10017	Determine necessity of stop advice	2011/10/18 13:47:06.950
e_7	10017	Check confirmation of receipt	2011/10/18 13:47:26.235
e_8	10017	Adjust confirmation of receipt	2011/10/18 13:47:41.811
e_9	10017	Check confirmation of receipt	2011/10/18 13:47:57.979
e_{10}	10017	Determine necessity to stop indication	2011/10/18 13:48:15.357
e_{11}	10017	Adjust confirmation of receipt	2011/10/18 13:48:30.632
e_{12}	10017	Check confirmation of receipt	2011/10/18 13:51:01.525
e_{13}	10017	Adjust confirmation of receipt	2011/10/18 13:56:57.603

2.2 Problem Formulation

The next-activity prediction problem consists in predicting the activity of a running trace. In our setting, the input consists of an event log—a set of traces with the attributes of their corresponding events.

Given an event log $\mathcal{L} = \{\sigma_i\}_{i=1}^N$ consisting of N traces, the online next-activity prediction scenario could be simulated in the following way.

A dataset $\mathcal{S} = \{(s_i, a_i)\}_{i=1}^M$, with $M > N$, is constructed from $\mathcal{L}$, where s_i is a sequence of events corresponding to a prefix σ_j^k of a trace $\sigma_j \in \mathcal{L}$, with $k \in \mathbb{N}^+$, and where a_i is the activity of the $(k+1)$ -th event of the trace σ_j . More formally, let $\sigma = \langle e_1, e_2, \dots, e_n \rangle$ be a trace, and $\Pi_{\mathcal{A}}(\sigma, k) = \pi_{\mathcal{A}}(e_k)$ the function returning the activity of the k -th event of σ , then:

$$\mathcal{S} = \{(s_i, a_i)\}_{i=1}^M = \bigcup_{j=1}^N \left\{ (\sigma_j^{k_j}, \Pi_{\mathcal{A}}(\sigma_j, k_j + 1)) \right\}_{k_j=1}^{|\sigma_j|-1}.$$

In particular, $\mathcal{S}$ is the dataset of all the prefixes of all the traces in $\mathcal{L}$ labelled with their next-activity in their corresponding trace.

2.3 Recurrent Neural Networks

Deep feed-forward neural networks, also called multi-layer perceptrons (MLPs), are layered deep NNs defining a mapping $\mathbf{y} = f(\mathbf{x}; \boldsymbol{\theta})$ from an input $\mathbf{x}$ space to an output $\mathbf{y}$ space—computing an higher level representation of the input—with parameters value $\boldsymbol{\theta}$ that result in the best function generating data approximation [15].

In a MLP each neuron of a layer is connected to all the neurons of the next layer without cycles. There are an input layer, an output layer, an many hidden layers. Learning deep neural networks requires computing the gradients of complicated functions, efficiently computed using the back-propagation algorithm.

Recurrent neural networks (RNNs) are NNs specialised for processing sequential data, processing all the symbols of the sequence and updating an internal fixed-length memory state $\mathbf{s}$ with the same transition matrix at each time step. Most recurrent networks are able to process sequences of variable length. The idea in RNNs is to have feedback connections thus proving a *parameters sharing* across different parts of the model.

Let $\mathbf{x}_t$ and $\mathbf{h}_t$ be, respectively, the input and the hidden state—the memory of the network—at time step t . In a RNN, the hidden state $\mathbf{h}_t$ is updated using the previous hidden state and the current input at each time step:

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{h}_{t-1} + \mathbf{b}),$$

where f is usually a non-linear function, such as $\tanh$, and the parameters are the bias vector $\mathbf{b}$ along with the weight matrices $\mathbf{U}$ and $\mathbf{W}$. Then, the output $\mathbf{o}_t$ at time t is computed as a function of the hidden state,

$$\mathbf{o}_t = f(\mathbf{V}\mathbf{h}_t + \mathbf{c}),$$

where $\mathbf{c}$ is the bias vector and $\mathbf{V}$ the weight matrix.

The most effective sequence models used in practical applications are called gated RNNs, including long short-term memory (LSTM) and networks based on the gated recurrent unit. A LSTM [16] is a special RNN architecture better suited for capturing long-term dependencies than vanilla RNNs. The core idea in LSTM is to introduce a cell state $\mathbf{C}_t$, more complex than the memory cell $\mathbf{h}_t$ in vanilla RNNs, where information is added or removed by gated structures, composed of a sigmoid neural network layer and a multiplication operation.

The LSTM model can be described by the following equations, where σ is the sigmoid function, and the operator $*$ denotes the Hadamard product—element-wise product:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f\mathbf{x}_t + \mathbf{V}_f\mathbf{h}_{t-1} + \mathbf{b}_f), \quad (1)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i\mathbf{x}_t + \mathbf{V}_i\mathbf{h}_{t-1} + \mathbf{b}_i), \quad (2)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o\mathbf{x}_t + \mathbf{V}_o\mathbf{h}_{t-1} + \mathbf{b}_o), \quad (3)$$

$$\mathbf{C}_t = \mathbf{f}_t * \mathbf{C}_{t-1} + \mathbf{i}_t * \tanh(\mathbf{W}_c\mathbf{x}_t + \mathbf{V}_c\mathbf{h}_{t-1} + \mathbf{b}_c), \quad (4)$$

$$\mathbf{h}_t = \mathbf{o}_t * \tanh(\mathbf{C}_t). \quad (5)$$

The first sigmoid layer $\mathbf{f}_t$ in Eq. 1—*forget gate*—operates on $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ and output numbers between 0 and 1 for each element in the cell state—deciding the information to throw away from the cell. Then, the sigmoid layer $\mathbf{i}_t$ in Eq. 2—*input gate*—decides the value to be updated, and the $\tanh$ layer in Eq. 4 proposes a vector of new candidate values to be added to the state. In Eq. 4 the new cell state $\mathbf{C}_t$ is obtained by forgetting some information and adding new scaled

information. The new value for $\mathbf{h}_t$ is computed in Eq. 5 by firstly deciding what part of the cell state to output as computed in Eq. 3—*output gate*.

As we can see from the above equations, learning a LSTM architecture is time consuming since it requires to learn many parameters—eight weight matrices $\mathbf{W}_f, \mathbf{W}_i, \mathbf{W}_o, \mathbf{W}_c$ and four bias vectors $\mathbf{b}_f, \mathbf{b}_i, \mathbf{b}_o, \mathbf{b}_c$.

3 The Proposed Inception Networks

Instead of using RNNs for the next-activity prediction problem, in this section we show how to use CNNs to tackle the same problem.

3.1 Convolutional Neural Networks

Convolutional neural networks [19] (CNNs) are a specialized kind of NNs for processing grid-like topology data, such as time-series data—one dimensional grid taking samples at regular time intervals—and image data—two dimensional grid of pixels. A convolutional layer convolves a kernel with the input in order to obtain a feature map. Often many different kernels for the same input are used, leading to many different feature maps. Different layers may have different kernel sizes. Between stacked convolutional layers, pooling layers—max or mean—are often inserted, greatly reducing the number of inputs thus speeding up the training process and reducing the number of parameters.

Convolutions are used over more than one axis at a time. For instance, if we have a two-dimensional input, we also want to use a two-dimensional kernel.

Given a two-dimensional input $\mathbf{X}$, the k -th feature map at location (i, j) in a given convolutional l -th layer is determined by the weight matrix $\mathbf{W}_k^l$ and the bias vector $\mathbf{b}_k^l$ with a non-linear activation function, such as the sigmoid function, as:

$$h_{i,j,k}^l = \sigma(\mathbf{W}_k^l * \mathbf{x}_{i,j}^l + \mathbf{b}_k^l),$$

where $\mathbf{x}_{i,j}^l$ is the input patch centered at location (i, j) of the l -th layer. The kernel $\mathbf{W}_k^l$ is shared for each possible location (i, j) , thus reducing the model complexity and making the network easier to train. The same process applies on a one-dimensional input using the convolutions on a single axis.

A pooling layer placed after a convolutional layer aims to achieve *shift-invariance* by reducing the resolution of the feature map $\mathbf{h}_{:, :, k}^l$ as:

$$y_{i,j,k}^l = \text{pool}(h_{m,n,k}^l), \forall (m, n) \in \mathcal{R}_{ij},$$

where $\mathcal{R}_{ij}$ is a local neighbourhood around location (i, j) .

3.2 Inception Networks

The Inception module, introduced in [22] as a building block for the GoogLeNet architecture, was an important milestone in the development of CNNs based classifiers. Since salient parts in images and sequences can have extremely large variation in size, choosing the right kernel size for the convolution operation becomes

difficult—large, respectively small, kernels are suited for globally, respectively locally, distributed information. The proposed solution is to have a convolutional layer with kernels with multiple sizes operating—an inception module. The network essentially would get a bit wider rather than deeper.

Figure 1a reports a naïve inception module for sequential data with three convolutions having kernel size 1, 2, and 3 respectively, plus an alternative parallel max-pooling path of size 3. The output of the module is the simple concatenation of the convolutions and pooling.

As these inception modules are stacked on top of each other (see Fig. 1b), their output correlation statistics are bound to vary [22].

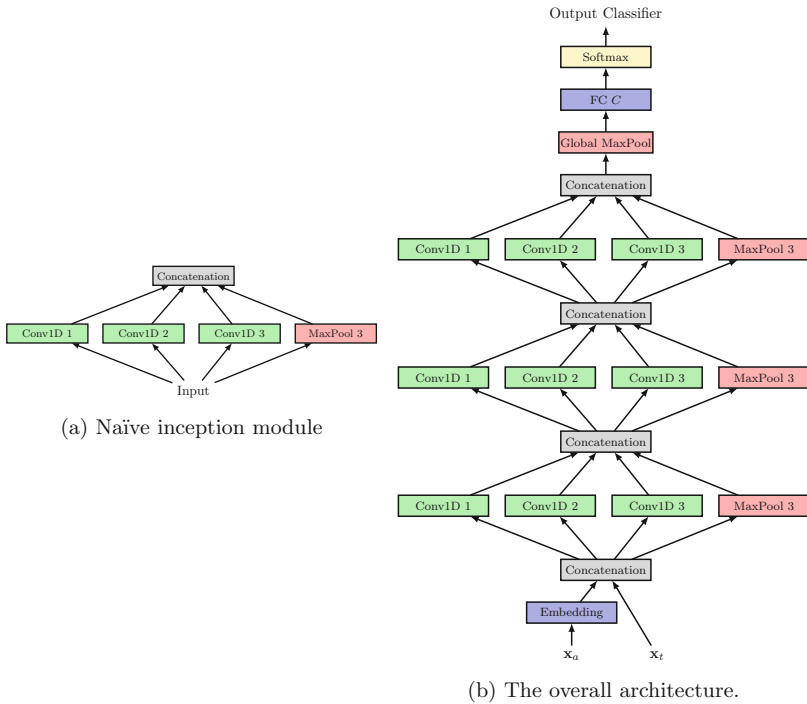


Fig. 1. The proposed inception architecture: (a) depicts the naïve inception module, while (b) is the stacking of three inception modules. Green boxes indicate the one-dimensional convolutions over the sequence, while red boxes indicate the pooling operations. (Color figure online)

3.3 Feature Representation

We recall that having an event log $\mathcal{L}$ we can obtain a dataset $\mathcal{S} = \{(s_i, a_i)\}_{i=1}^M$, as described in Sect. 2.2, consisting of all the prefixes s_i of all the traces in $\mathcal{L}$, labelled with their next-activity a_i in their corresponding trace.

However, since $\mathcal{S}$ consists of prefixes and activities denoted with categorical values, they should be converted into a numerical representation $\mathbf{x} \in \mathbb{R}^l$, with a given $l \in \mathbb{N}^+$, in order to be processed by our NN architecture.

For each sample $(s_i, a_i) = (e_{i1}, e_{i2} \dots, e_{ik_i}), a_i) \in \mathcal{S}$, we can construct its corresponding numerical representation, denoted as $((\mathbf{x}_{\text{act}}^i, \mathbf{x}_{\text{t}}^i), y^i)$, in the following way.

The vector $\mathbf{x}_{\text{act}}^i \in \mathbb{N}^k$ corresponds to the numerical representation of the activities sequence in s_i . Let $f_c : \mathcal{A} \rightarrow [1, 2, \dots, C]$, being $C = |\mathcal{A}|$ the cardinality of the activity-vocabulary—the number of classes—be a function assigning a numerical value to each activity, then $\mathbf{x}_{\text{act}}^i = (x_{a1}^i, x_{a2}^i, \dots, x_{ak_i}^i)$ is the vector where each $x_j^i = f_c(\pi_A(e_{ij}))$, $1 \leq j \leq k_i$. In the same way we can encode the label as $y^i = f_c(a_i)$. As usual in deep learning, in order to use losses such as the categorical cross-entropy, the class vector $\mathbf{y}$ should be converted to a binary class matrix $\mathbf{Y}$, i.e., each row $\mathbf{y}_i \in \mathbf{Y}$ is the one-hot encoding of the label y_i .

The vector $\mathbf{x}_{\text{t}}^i = (x_{t1}^i, x_{t2}^i, \dots, x_{tk_i}^i) \in \mathbb{R}^k$ is the numerical representation of the temporal information of the activities. In particular, for each event $e_{ij} \in s_i$, $1 \leq j \leq k_i$, $x_{tj}^i = \pi_{\mathcal{T}}(e_{ij}) - \pi_{\mathcal{T}}(e_{i(j-1)})$ represents the time difference from the previous event—it is zero for the first event in the sequence.

Example 1 (Feature representation). Let $s = (e_5, e_6, e_7, e_8)$ be a prefix, as reported in Table 1, consisting of 4 activities. Let f_c be defined with the mappings { “Confirmation of receipt” $\rightarrow 0$, “Determine necessity of stop advice” $\rightarrow 1$, “Check confirmation of receipt” $\rightarrow 2$, “Adjust confirmation of receipt” $\rightarrow 3, \dots$ }, then $\mathbf{x}_{\text{act}} = (0, 1, 2, 3)$ and $\mathbf{x}_{\text{t}} = (0.0, 27.271, 19.285, 15.576)$.

Thus, given the dataset $\mathcal{S} = \{(s_i, a_i)\}_{i=1}^M$ in a categorical representation we obtained its numerical representation $\mathcal{D} = (\mathbf{X}, \mathbf{y}) = \{(\mathbf{x}_{\text{act}}^i, \mathbf{x}_{\text{t}}^i), y^i\}_{i=1}^M$. Since all the sequences in $\mathcal{S}$ could not have the same length, then all the obtained sequences $\mathbf{x}_{\text{act}}^i$ and $\mathbf{x}_{\text{t}}^i$ have been pre-padded with the value 0.0 to the same length $\max_{i=1, \dots, M} |\mathbf{x}_{\text{act}}^i|$ before to feed them to the NN architectures.

3.4 Architecture Details

Given the training dataset $\mathcal{D} = \{(\mathbf{x}_{\text{act}}^i, \mathbf{x}_{\text{t}}^i), y^i\}_{i=1}^M$, the NN architectures have two inputs $\mathbf{x}_{\text{act}}^i \in \mathbb{N}^k$ and $\mathbf{x}_{\text{t}}^i \in \mathbb{R}^k$. Categorical values in $\mathbf{x}_{\text{act}}$ should be represented using a one-hot-encoding approach, as done in [23], that however could lead to large sparse vectors. Inspired by the neural language modeling works [4]—each word is mapped to a fixed size vector in $\mathbb{R}^d$, i.e., an *embedding*— $\mathbf{x}_{\text{act}}^i$ is the input to an embedding layer—jointly learned with the model—with size equal to $d = \lceil C/2 \rceil$. While $\mathbf{x}_{\text{t}}^i$ is simply concatenated to the output of the embedding layer whose values are in $\mathbb{R}^{k \times d}$, leading to a final input representation in $\mathbb{R}^{k \times d + 1}$.

Then many inception modules, whose basic structure is reported in Fig. 1a, are stacked in order to learn increasingly complex patterns.

Each inception module applies three one-dimensional convolutional layers on the same input with different kernel sizes—resp., 1, 2, and 3. A max-pooling layer is applied on the same input. Finally, both the convolutional and pooling layer outputs are concatenated to obtain the inception output.

The last inception output in the stack is the input to a global max pooling layer, whose output is used by a final fully connected layer outputting the classification probabilities. The output probabilities have been computed using the *softmax* activation function:

$$p_i = \frac{\exp(e_i)}{\sum_{j=1}^C \exp(e_j)}.$$

The non-linear activation function we chose to use is the Rectifier Linear Unit (ReLU) [14] computing $\max(0, x)$. The ReLU activation function limits the gradient vanishing problem as its derivative is always one when x is positive. Let Θ denote the parameters of the network, the optimum parameters have been obtained by minimizing the mean-squared-error loss function:

$$\mathcal{L} = \frac{1}{M} \sum_{i=1}^M \ell_{\text{mse}}(\Theta; \mathbf{y}_i, \mathbf{p}_i).$$

4 Experimental Evaluation

Here we present the empirical results of our proposed neural network architecture when compared to LSTM models. For LSTM architectures the same implementation adopted in [23] has been used¹. However, for a fair comparison the original code for LSTM architectures have been slightly modified in the following way. In [23] the architecture takes as input the activities $\mathbf{x}_{\text{act}}$ only. Here the input has been extended to consider both $\mathbf{x}_{\text{act}}$ and $\mathbf{x}_{\text{t}}$. Furthermore, in [23] the categorical values in $\mathbf{x}_{\text{act}}$ have been one-hot-encoded, while here the previous embedding approach has been adopted.

4.1 Experimental Setup

We evaluated the performance of our proposed approach in predicting the next activity on three event logs. For each log, we used a 3-fold cross validation approach. The performance measures we adopted to assess the model performance are the Brier score [7] and the accuracy. In particular, the brier score can be interpreted as a measure of the calibration of a set of probabilistic predictions, measuring error of the predicted likelihoods over all symbols—the lower the Brier score is for a set of predictions, the better the predictions are calibrated.

The adopted logs—Table 3 reports statistics in terms of number of classes, traces, and resulting sequences—are the following:

¹ The original code used in [23] is available at <https://github.com/TaXxER/rnnalpha>.

- *Receipt phase*²: this log originates from the CoSeLoG project executed under NWO project number 638.001.211. It contains the records of the execution of the receiving phase of the building permit application process in an anonymous municipality in the Netherlands;
- *bpi12*³: this log describes a loan application process. It has been pre-processed as reported in [24];
- *helpdesk*⁴: this log contains events from a ticketing management process of the help desk of an Italian software company.

Table 2. Hyperparameter search space for our method and LSTM adopted for the TPE procedure.

OUR		LSTM	
layers	{1, 2, 3}	layers	{1, 2, 3}
batch_size	{2 ⁹ , 2 ¹⁰ }	batch_size	{2 ⁹ , 2 ¹⁰ }
learning_rate	[0.00001, 0.01]	dropout	[0, 0.5]
		layer_size	[10, 150]
		learning_rate	[0.00001, 0.01]
		l_1	[0.00001, 0.01]
		l_2	[0.00001, 0.01]

For both our method and LSTM we conducted a hyper-parameter optimization by using the 20% of the training set as a validation set. We chose the configuration of the parameter that achieved the best validation loss. The hyper-parameters and their corresponding possible values are reported in Table 2. We employed the hyper-parameter optimization tree-structured Parzen estimator (TPE) as proposed in [5].

The weights are initialized following the Xavier scheme [13]. For all deep learning approaches, the weights of the neural networks have been optimized using Adam [17] minimizing the mean squared error, a variant of the stochastic gradient descent. An early stopping approach has been used for regularization—the training phase is stopped when there is no improvement of the loss on the validation set for 20 consecutive epochs. A limit on the number of epochs has been set to 200. The epoch with best validation loss was selected and the corresponding model was evaluated on the test set.

Both our proposed approach and the LSTM architecture have been implemented in Python using the Keras⁵ and Tensorflow⁶ libraries⁷.

² <https://doi.org/10.4121/uuid:a07386a5-7be3-4367-9535-70bc9e77dbe6>.

³ <https://doi.org/10.4121/uuid:a07386a5-7be3-4367-9535-70bc9e77dbe6>.

⁴ <https://doi.org/10.17632/39bp3vv62t.1>

⁵ <https://keras.io/>.

⁶ <https://www.tensorflow.org/>.

⁷ Source code available at <https://github.com/nicoladimauro/nnpm>.

All the experiments have been executed on a single GeForce GTX TITAN X GPU—execution time are reported in Table 5.

Table 3. Dataset statistics.

	Classes	Cases	Sequences
Receipt phase	26	1434	7143
helpdesk	8	3804	9906
bpi12	22	13087	151419

4.2 Results

As we can see from Table 4, our proposed approach outperform the LSTM one both in terms of Brier score and accuracy on all the adopted datasets. The first column in Table 4 reports the accuracy of a random predictor, guessing a label with uniform probability. The value of the Brier score are very low, corresponding to a low error of the predicted likelihoods over the symbols. This is confirmed from the accuracy scores that are always greater than those obtained using LSTM architectures. Specifically, our proposed approach obtain, when compared to the LSTM model based one, an increase of the accuracy of 12.17% on average (34.41% on *Receipt phase*, 2.34% on *helpdesk*, and 3.95% on *bpi12*). Even the Brier score is always improved on each considered dataset, decreasing of 15.38% on average (44.44% on *Receipt phase*, 4.4% on *helpdesk*, and 7.14% on *bpi12*).

Table 4. The Brier score and the classification accuracy (mean $\pm$ standard deviation on three folds) for our proposed network and LSTM on all the event logs.

	Brier score			Accuracy	
	RAND	OUR	LSTM	OUR	LSTM
Receipt	0.037 $\pm$ 0	0.010 $\pm$ 0.000	0.018 $\pm$ 0.012	0.832 $\pm$ 0.005	0.619 $\pm$ 0.365
helpdesk	0.109 $\pm$ 0	0.043 $\pm$ 0.001	0.045 $\pm$ 0.002	0.785 $\pm$ 0.005	0.767 $\pm$ 0.015
bpi12	0.043 $\pm$ 0	0.013 $\pm$ 0.000	0.014 $\pm$ 0.002	0.789 $\pm$ 0.001	0.759 $\pm$ 0.029
Average		0.022	0.026	0.802	0.715

Another interesting aspect of our proposed solution, as expected when compared to recurrent networks, regards the computational time for learning the network parameters. Indeed, it is well known that RNNs are slow to be learnt.

As we can see from Table 5, even if on some datasets, such as *helpdesk* and *bpi12*, the number of parameters of our best architecture is greater than that of the LSTM based one, the time spent in seconds to optimize them is always smaller than that used to optimize the LSTM parameters—2 times faster on average.

We can conclude from this experimental evaluation that our proposed neural network architecture is more accurate, both in term of calibrated predictions and accuracy, and more efficient than LSTM in predicting the next-activity of a running execution trace of a business process.

Table 5. The number of parameters and learning time spent in seconds (mean on three folds) of our proposed network and LSTM on all the event logs.

	# parameters		Time	
	OUR	LSTM	OUR	LSTM
Receipt phase	14.2 k	39.4 k	10.7	19.2
helpdesk	35.5 k	22.0 k	17.6	25.3
bpi12	48.4 k	36.4 k	265.1	604.7
Average			97.8	216.4

5 Conclusion

Predicting the next activity of a running execution trace of a business process represents a challenging task in process mining [20]. The problem has been successfully tackled using recurrent neural networks providing state-of-the-art results [23]. However, recurrent networks are time consuming and in many cases they have been outperformed by convolutional network when dealing with sequential data, as reported in [3].

In this paper we proposed a deep neural network model based on convolutional neural networks to tackle the problem of next-activity prediction. In particular, we proposed an inception architecture, similar to that used for computer vision, but adapted for sequential data. When compared to a LSTM recurrent neural network on different real-world dataset, the results of the proposed approach prove its validity both in terms of accuracy and computational complexity.

A possible extension of the proposed approach should be to predict at the same time both the next-activity and the its execution time as already done in [24]. Extensions of the naïve inception module to more complex ones are also possible to be investigated.

Acknowledgments. This research is partially funded by the *Knowledge Community for Efficient Training through Virtual Technologies* Italian project (KOMETA, code 2B1MMF1), under the program POR Puglia FESR-FSE 2014–2020 - Asse prioritario 1 - Ricerca, sviluppo tecnologico, innovazione - SubAzione 1.4.b - BANDO INNOLABS supported by Regione Puglia, as well as by the *Electronic Shopping & Home delivery of Edible goods with Low environmental Footprint* Italian project (ESHELF), under the Apulian INNONETWORK programme.

References

1. van der Aalst, W.M.P.: Process Mining - Data Science in Action, 2nd edn. Springer, Heidelberg (2016). <https://doi.org/10.1007/978-3-662-49851-4>
2. Appice, A., Mauro, N.D., Malerba, D.: Leveraging shallow machine learning to predict business process behavior. In: SCC, pp. 184–188 (2019)
3. Bai, S., Kolter, J.Z., Koltun, V.: Convolutional sequence modeling revisited. In: ICLR (2018)
4. Bengio, Y., Ducharme, R., Vincent, P., Janvin, C.: A neural probabilistic language model. *JMLR* **3**, 1137–1155 (2003)
5. Bergstra, J., Bardenet, R., Bengio, Y., Kégl, B.: Algorithms for hyper-parameter optimization. In: NIPS, pp. 2546–2554 (2011)
6. Breuker, D., Matzner, M., Delfmann, P., Becker, J.: Comprehensible predictive models for business processes. *J. MIS Q.* **40**, 1009–1034 (2016)
7. Brier, G.W.: Verification of forecasts expressed in terms of probability. *Mon. Weather Rev.* **78**(1), 1–3 (1950)
8. Cho, K., et al.: Learning phrase representations using RNN encoder–decoder for statistical machine translation. In: EMNLP, pp. 1724–1734 (2014)
9. Chorowski, J.K., Bahdanau, D., Serdyuk, D., Cho, K., Bengio, Y.: Attention-based models for speech recognition. In: NIPS, pp. 577–585 (2015)
10. Devlin, J., Chang, M., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. CoRR abs/1810.04805 (2018)
11. Evermann, J., Rehse, J.-R., Fettke, P.: A deep learning approach for predicting process behaviour at runtime. In: Dumas, M., Fantinato, M. (eds.) BPM 2016. LNBP, vol. 281, pp. 327–338. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-58457-7_24
12. Di Francescomarino, C., Ghidini, C., Maggi, F.M., Petrucci, G., Yeshchenko, A.: An eye into the future: leveraging a-priori knowledge in predictive business process monitoring. In: Carmona, J., Engels, G., Kumar, A. (eds.) BPM 2017. LNCS, vol. 10445, pp. 252–268. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-65000-5_15
13. Glorot, X., Bengio, Y.: Understanding the difficulty of training deep feedforward neural networks. In: AISTATS, pp. 249–256 (2010)
14. Glorot, X., Bordes, A., Bengio, Y.: Deep sparse rectifier neural networks. In: AISTATS, vol. 15, pp. 315–323. PMLR (2011)
15. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge (2016)
16. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)
17. Kingma, D., Ba, J.: Adam: a method for stochastic optimization. In: ICLR (2014)
18. Krizhevsky, A., Sutskever, I., Hinton, G.E.: ImageNet classification with deep convolutional neural networks. *CACM* **60**(6), 84–90 (2017)
19. LeCun, Y., Haffner, P., Bottou, L., Bengio, Y.: Object recognition with gradient-based learning. In: Shape, Contour and Grouping in Computer Vision. LNCS, vol. 1681, pp. 319–345. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-46805-6_19
20. Polato, M., Sperduti, A., Burattin, A., de Leoni, M.: Time and activity sequence prediction of business process instances. *Computing* **100**(9), 1005–1031 (2018)
21. Schmidhuber, J.: Deep learning in neural networks: an overview. *Neural Netw.* **61**, 85–117 (2015)

22. Szegedy, C., et al.: Going deeper with convolutions. In: CVPR (2015)
23. Tax, N., Teinemaa, I., van Zelst, S.J.: An interdisciplinary comparison of sequence modeling methods for next-element prediction. CoRR abs/1811.00062 (2018)
24. Tax, N., Verenich, I., La Rosa, M., Dumas, M.: Predictive business process monitoring with LSTM neural networks. In: Dubois, E., Pohl, K. (eds.) CAiSE 2017. LNCS, vol. 10253, pp. 477–492. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-59536-8_30